

FSS · DATA MAPPING TOOL

Gen Mapping Tool

Tự động sinh ETL Mapping

Từ DataModel YAML + BRD Spec → Mapping YAML sẵn sàng cho ETL Pipeline

gen_mapping_atomic.py · Tầng hiện tại: STG → Atomic

TỔNG QUAN

Tự động hóa sinh Mapping YAML cho các tầng

QUY TẮC ĐẶT TÊN FILE

ĐỌC

`dm_<layer>_<tên bảng đích>-<phân hệ>.<tên bảng nguồn>.yaml` →

SINH

`mapping_<layer>_<tên bảng đích>-<phân hệ>.<tên bảng nguồn>.yaml`

Ví dụ: `dm_atm_fnd_mgt_co-FIMS.FUNDCOMPANY.yaml` → `mapping_atm_fnd_mgt_co-FIMS.FUNDCOMPANY.yaml`

 Mục tiêu

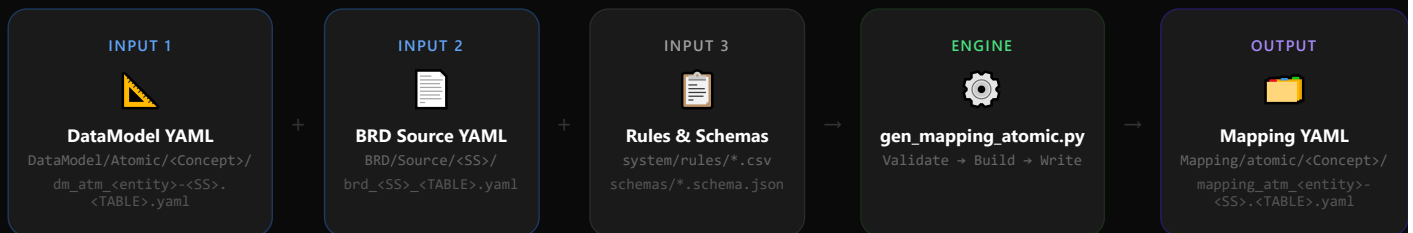
- Tự động hóa sinh mapping YAML, giảm lỗi thủ công
- Đảm bảo nhất quán DataModel ↔ BRD ↔ Mapping
- Mỗi tầng có script riêng, mở rộng dễ dàng

 Cấu hình hiện tại — tầng **STG → ATM**

- Đọc: `dm_atm_*.yaml` + BRD YAML
- Sinh: `mapping_atm_*.yaml`
- Layer cố định: `stg-atm`

LUỒNG DỮ LIỆU

Input → Processing → Output



Tên file output: thay prefix `dm_` → `mapping_`, giữ nguyên phần còn lại

ĐẦU VÀO

12 Source Systems được hỗ trợ

KNT

Kiểm tra thuế

ECAT

Danh mục điện tử

FIMS

Quản lý quỹ

FMSFund
Management**GSGD**

Giám sát giao dịch

IDS

Phát hành CK

MDDS

Market Data

NHNCKNghiệp vụ hành
nghề**ORDER
TRADE**

Lệnh giao dịch

QLRR

Quản lý rủi ro

SCMS

Công ty CK

THANHTRA

Thanh tra giám sát

Parse từ tên file: dm_atm_fnd_mgt_co-**FIMS**.FUNDCOMPANY.yaml → Source System = **FIMS**

SỬ DỤNG

4 Chế độ CLI thực thi

Batch toàn bộ

Xử lý tất cả `dm_atm_*.yaml` trong `DataModel/Atomic/` đệ quy

```
python gen_mapping_atomic.py
```

Một file

Xử lý một file DataModel cụ thể

```
python gen_mapping_atomic.py dm_atm_fnd_mgt_co-FIMS.FUNDCOMPANY.yaml
```

Glob pattern

Lọc nhiều file theo pattern

```
python gen_mapping_atomic.py "dm_atm_*-FIMS.*.yaml"
```

Folder mode

Xử lý tất cả file trong một Concept folder (phải nằm dưới `DataModel/`)

```
python gen_mapping_atomic.py --folder DataModel/Atomic/Involved_Party
```

Chỉ chấp nhận file có prefix `dm_atm_` · Layer cố định: `ods-atm`

INPUT VALIDATION

Kiểm tra Schema trước khi xử lý



❌ Lỗi Blocking

- Validation fail → bỏ qua file, không sinh output
- BRD entry không tìm thấy → ghi lỗi vào summary
- Output không hợp lệ → không ghi file

✅ Warn-and-Continue

- source_column = null → cảnh báo, tiếp tục
- BRD columns file thiếu → cảnh báo, tiếp tục
- FK comment thiếu SSC → cảnh báo, fallback

UTILITY MODULE

Alias Generator — Đặt tên ngắn gọn

1

Override Dict

Kiểm tra 6 alias cố định trước tiên

2

Multi-Word

Có underscore → 2 ký tự đầu mỗi segment

3

wordninja

Single-word → tách từ, nhóm ≥4 ký tự

4

Fallback

4 ký tự đầu nếu không tách được

TABLE NAME	LOẠI	ALIAS	GHI CHÚ
SECBUSINES	Override	se_bu	Từ override dict
AA_ARRANGEMENT_LINKED	Multi-word	aa_ar_li	2 chars mỗi segment
INVESACC	wordninja	in	inves+acc → in+ac → in
FUNDCOMPANY	wordninja	fu_co	fund+company → fu_co

Xung đột trong cùng source system → thêm suffix số: co_ta → co_ta_2

CORE LOGIC

Transformation Rule Engine — 7 mức ưu tiên

- 1 Source System Code attribute
'<SSC>' – literal từ `classification_context`
- 2 Surrogate Key — Primary Key
{hash_id}('<own_SSC>', alias.<code_sibling_source_column>)
- 3 Surrogate Key — Foreign Key
{hash_id}('<target_SSC>', alias.<source_column>) – SSC parse từ FK comment
- 4 Classification Value + "SOURCE_SYSTEM" trong comment
'<SSC>' – literal
- 5 Domain Array<Text> hoặc Array<Struct>
<cte_alias>.<physical_name> – tham chiếu CTE output
- 6 Có source_column
<alias>.<source_column>
- 7 Không có source_column
null

Rule đầu tiên khớp được áp dụng — các rule thấp hơn bị bỏ qua

OUTPUT STRUCTURE

Regular Entity — Cấu trúc Mapping YAML

 Header & Target

```
schema_type: mapping
mapping.id: MAP-ATM-{entity}-{SS}.{TABLE}
mapping.dm_ref: ATM-{entity}-{SS}.{TABLE}
mapping.brd_ref: BRD-SRC-{SS}-{TABLE}
target.namespace: atm
target.etl_handle: SCD4A | SCD2 | Upsert | Fact Append
```

 Input & Mapping Columns

```
input:
- source_type: physical_table
namespace: ods
alias: in # auto-generated
filter: # từ BRD filter_logic
mapping_columns:
- target_column: entity_id
transformation: {hash_id}('SSC', in.Id)
- target_column: src_stm_code
transformation: 'FMS_INVESACC'
```

- SCD4A — Fundamental
- SCD2 — Relative
- Upsert — Classification
- Fact Append

INPUT SECTION

Physical Table & Derived CTE

Physical Table

- `source_type: physical_table`
- `namespace: ods` (từ layer param)
- `filter` : lấy từ BRD `filter_logic`
- `select_fields` : danh sách cột cụ thể
- `alias` : tự sinh theo Alias Generator

Derived CTE (Junction)

- `source_type: derived_cte`
- `namespace: null`
- `filter: GROUP BY {FK_col}` nếu aggregate
- Dùng cho domain `Array<Text>` / `Array<Struct>`
- `COLLECT_LIST(NAMED_STRUCT(...))`

VÍ DỤ JUNCTION TABLE CHO ARRAY<STRUCT>

```
# Input 1 – physical table (raw)
alias: fu_bu_raw    select_fields: "FundId, Id"
# Input 2 – derived CTE (aggregated)
alias: fu_bu        filter: "GROUP BY FundId"
select_fields: COLLECT_LIST(NAMED_STRUCT('business_type_id', hash_id('SSC', fu_bu_raw.Id), ...))
```

SPECIAL CASE

Alternative Identification — `unpivot_cte`

⚡ Pattern: UNPIVOT nhiều loại giấy tờ

- 1 bảng nguồn chứa **nhiều cột** giấy tờ khác nhau
- Cần **UNPIVOT** thành nhiều dòng, mỗi dòng = 1 loại giấy tờ
- Mỗi `leg` = 1 loại + tập cột nguồn tương ứng

Ví dụ từ `mapping_atm_ip_alt_identn-FIMS.BANKMONI.yaml`:

`BUSINESS_LICENSE`

← `IdNo, IdDate, IdAdd`

`OPERATING_LICENSE` ← `RegNo, RegDate, RegAdd`

🔗 Cấu trúc input — `unpivot_cte`

```
input:
- source_type: physical_table
alias: ba_mo
select_fields: Id, IdNo, IdDate, IdAdd, RegNo...
- source_type: unpivot_cte
table_name: ba_mo
alias: unpvt
unpivot_spec:
type_column: identn_tp_code
legs:
- type_value: BUSINESS_LICENSE
source_columns: [IdNo, IdDate, IdAdd]
- type_value: OPERATING_LICENSE
source_columns: [RegNo, RegDate, RegAdd]
```

`mapping_columns` dùng alias `unpvt` như bình thường — `unpvt.identn_tp_code, unpvt.identn_nbr...`

KIẾN TRÚC

Các Module thành phần



CLI Entry Point

gen_mapping_atomic.py
4 execution modes
Layer: stg-atm



Input Validator

input_validator.py
JSON Schema validation
BRD + DM files



Alias Generator

alias_generator.py
Override → Multi-word
→ wordninja → first-4



Rule Engine

transformation_rules.py
7-priority chain
hash_id · literal · null



Atomic Generator

atomic_mapping_generator.py
Header · Target · Input
Relationship · Columns



Shared Entity Handler

mapping_legs structure
Per-source file
etl_derived_value



Execution Reporter

execution_reporter.py
ERROR · WARNING
Summary report



Config & File I/O

config.py
Path resolution
Source system parsing

BÁO CÁO

Execution Summary Report

Mẫu output

```
EXECUTION SUMMARY

ERRORS (blocking):
- dm_xyz-FIMS.TABLE.yaml: BRD not found

WARNINGS (non-blocking):
- dm_foo-FMS.BAR.yaml: col_x source=null

Total processed: 15
Success: 12
Errors: 2
With warnings: 3
```

Bất biến số liệu

success + error = total

Tính nhất quán đếm file

✖ ERROR

Blocking — không sinh output

⚠ WARNING

Non-blocking — sinh output nhưng cần xem

3 loại warning: null source_column · missing BRD columns file · FK comment thiếu SSC

KIỂM THỬ

Testing Strategy — 21 Properties

 Property-Based Tests

- Dùng thư viện `hypothesis`
- Tối thiểu 100 iterations/test
- 21 properties bao phủ toàn bộ spec
- Source system validation
- Alias uniqueness invariant
- Summary counts arithmetic

 Unit Tests

- Override dict aliases (6 entries)
- Header format strings
- Transformation rules (1 example/rule)
- Shared entity leg extraction
- BRD filter_logic propagation
- Summary report formatting

 Integration Tests

- End-to-end với sample DM + BRD
- Batch: valid + invalid files
- Output validates vs schema
- Directory creation khi thiếu
- Missing BRD entry detection
- YAML round-trip validation

tests/ → test_alias_generator · test_transformation_rules · test_input_validator · test_atomic_mapping_generator · test_execution_reporter

TIẾN ĐỘ

Trạng thái **Implementation**

#	MODULE	TRẠNG THÁI	GHI CHÚ
1	config.py — File I/O, path resolution	Done	parse_source_system, datamodel_to_mapping
2	input_validator.py — JSON Schema	Done	BRD + DM + output validation
3	alias_generator.py	Done	wordninja + override dict
4	transformation_rules.py	Done	FK SSC from comment + warning
5	atomic_mapping_generator.py	Done	Array<Struct> junction, COLLECT_LIST
6	execution_reporter.py	Done	ERROR · WARNING · Summary
7	gen_mapping_atomic.py CLI	Done	4 modes, layer=ods-atm
11	FK SSC + Pure junction COLLECT_LIST	Done	Req 11 — enhancements

TỔNG KẾT

Gen Mapping Tool — Giá trị cốt lõi



Tự động hóa 100%

Từ DataModel YAML + BRD spec → sinh mapping YAML hoàn chỉnh, không cần viết tay



Validate 2 chiều

Kiểm tra input (DM + BRD) và output đều theo JSON Schema — đảm bảo mapping luôn hợp lệ



Rule Engine 7 cấp

Priority chain đảm bảo mỗi attribute nhận đúng transformation — hash_id, literal, CTE, hay null



4 CLI Modes

Batch toàn bộ, single file, glob pattern, folder mode — linh hoạt cho mọi workflow



Execution Report

ERROR & WARNING phân loại rõ ràng, summary counts đảm bảo tính nhất quán



21 Properties Tested

hypothesis property-based tests đảm bảo đúng với mọi input hợp lệ, không chỉ sample cases

12 source systems · 3 shared entities · SCD4A / SCD2 / Upsert / Fact Append